

Appunti di Information Retrieval

Anno Accademico 2026/2027

Indice

1	Introduzione e Architettura dell'Information Retrieval	1
1.1	Definizione e Motivazioni	1
1.1.1	Dai Motori Web all'Agentic Search	1
1.1.2	Information Retrieval vs. Data Retrieval (Basi di Dati)	2
1.2	I Due Macro-Obiettivi: Efficacia ed Efficienza	3
1.3	Architettura di un Sistema di Information Retrieval	3
1.3.1	Pipeline di Text Processing	4
1.4	Proprietà del Linguaggio e Modelli di Reperimento	4
1.5	L'Inverted Index (Indice Invertito)	5
1.6	Tecniche di Compressione dell'Indice	7
1.6.1	Binary Interpolative Coding	7
1.6.2	Compressione Basata su Dizionario: LZ77	7
1.7	Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità	7
1.8	Evoluzione verso il Neural Information Retrieval	8
2	Valutazione dei Sistemi di Information Retrieval	10
2.1	Architettura di un Motore di Ricerca	10
2.2	Come Valutare la Qualità di un Motore di Ricerca	12
2.3	Misurare la Soddisfazione dell'Utente	13
2.4	Il Paradigma di Cranfield e le Collezioni Benchmark	15
2.4.1	La Sfida della Scalabilità Combinatoria	15
2.4.2	Origini delle Query di Test	16
2.4.3	Crowdsourcing: Opportunità e Limiti	16
2.4.4	L'Infrastruttura TREC (<i>Text REtrieval Conference</i>) . .	17
2.4.5	La Tecnica del Pooling	17
2.5	Bisogno Informativo vs. Query	19
2.6	Metriche di Valutazione Non Ordinate (<i>Unranked Measures</i>) .	19
2.6.1	Precisione, Richiamo e loro Trade-off	20
2.6.2	La Media Armonica: F-Measure (F_1 -Score)	21
2.6.3	La Misura F_β Pesata	22

2.7	Metriche Basate sul Ranking (<i>Ranked Measures</i> - Rilevanza Binaria)	22
2.7.1	Precision@k ($P@k$) e Recall@k ($R@k$)	22
2.7.2	Average Precision (AP)	23
2.7.3	Mean Average Precision (MAP)	25
2.7.4	Metriche Troncate per il Web e Recommender Systems: AP@k e MAP@k	26
2.7.5	Mean Reciprocal Rank (MRR)	27
2.8	Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG . . .	28
2.8.1	Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)	29
2.8.2	Normalized Discounted Cumulative Gain (NDCG) . . .	30
2.8.3	Esempio Numerico Completo: Calcolo Passo-Passo . .	30
2.8.4	Confronto tra Funzioni di Ranking Alternative	31
3	Efficienza Algoritmica sulle Architetture CPU Moderne	33
3.1	Il Ruolo dell'Efficienza nell'Information Retrieval	33
3.1.1	I Tre Pilastri dell'Efficienza nei Sistemi IR	33
3.2	La Gerarchia di Memoria e i Principi di Località	34
3.2.1	Il Divario di Latenza tra CPU e Memoria	34
3.2.2	Architettura Fisica sul Chip	35
3.2.3	Località Temporale e Spaziale	36
3.2.4	Hardware Prefetcher	36
3.3	Caso di Studio: Scansione Sequenziale contro <i>Pointer Chasing</i> .	37
3.3.1	Analisi delle Cause Fisiche	38
3.4	Forme di Parallelismo e Architettura della Pipeline	40
3.4.1	Anatomia del Pipelining a Cinque Stadi	40
3.4.2	Latenza, Throughput e Profondità della Pipeline	41
3.5	Esecuzione Superscalare, Interleaving e Conflitti Strutturali . .	41
3.5.1	L'Esperimento dell' <i>Interleaving</i> Aritmetico	42
3.5.2	Sovrapposizione delle Istruzioni e Saturazione delle Porte .	43
3.6	Data Hazard e Dipendenze Seriali	44
3.6.1	Soluzione Algoritmica: <i>Interleaving</i> di Vettori Indipendenti	45
3.7	Control Hazard, Esecuzione Speculativa e Branch Prediction .	45
3.7.1	Il Paradosso della Condizione Casuale	46
3.7.2	Verifica Sperimentale: Ordinamento dei Dati e Pattern Periodici	47
3.7.3	Tecniche <i>Branchless</i> e il Salto Implicito dei Cicli <code>for</code> .	48
3.8	Sintesi: dai Vincoli Hardware alle Scelte Algoritmiche	48

Capitolo 1

Introduzione e Architettura dell'Information Retrieval

1.1 Definizione e Motivazioni

L'**Information Retrieval** (IR, *Reperimento delle Informazioni*) è la disciplina scientifica e ingegneristica che studia la rappresentazione, la memorizzazione, l'organizzazione e l'accesso a collezioni di dati prevalentemente non strutturati o semi-strutturati, al fine di soddisfare un **bisogno informativo** (*information need*) espresso da un utente [1, 2].

Definizione 1.1.1 (Information Retrieval (Manning et al.)). L'Information Retrieval consiste nel reperire materiale di natura prevalentemente non strutturata (solitamente documenti testuali) che soddisfa un bisogno informativo all'interno di collezioni di grandi dimensioni memorizzate su calcolatori elettronici.

L'IR non si limita ai soli motori di ricerca Web generalisti (Google, Bing), ma costituisce l'infrastruttura abilitante di e-commerce e marketplace, enterprise/desktop search, sistemi **RAG** (*Retrieval-Augmented Generation*, che alimentano i Large Language Model con documenti esterni contestualmente rilevanti) e motori di raccomandazione.

1.1.1 Dai Motori Web all'Agentic Search

L'avvento dei modelli linguistici conversazionali (LLM) non ha reso obsoleto l'Information Retrieval, ma ne ha ampliato l'impatto: un LLM moderno si appoggia costantemente ai principi cardine dell'IR (comprensione semantica della query, matching e ranking, reperimento contestuale) per superare il

limite del *knowledge cutoff*. Nel paradigma **RAG**, il sistema intercetta la richiesta dell'utente, interroga un motore IR, inietta i documenti estratti nel contesto dell'LLM e genera una risposta supportata da citazioni verificabili.

Definizione 1.1.2 (Agentic Search). L'**Agentic Search** è un processo di reperimento dinamico in cui un agente intelligente determina in modo autonomo quali ricerche successive condurre, basandosi sulle evidenze parziali raccolte: ispeziona le fonti, individua lacune conoscitive (*information gaps*), formula nuove query di raffinamento e reitera il ciclo fino alla convergenza informativa.

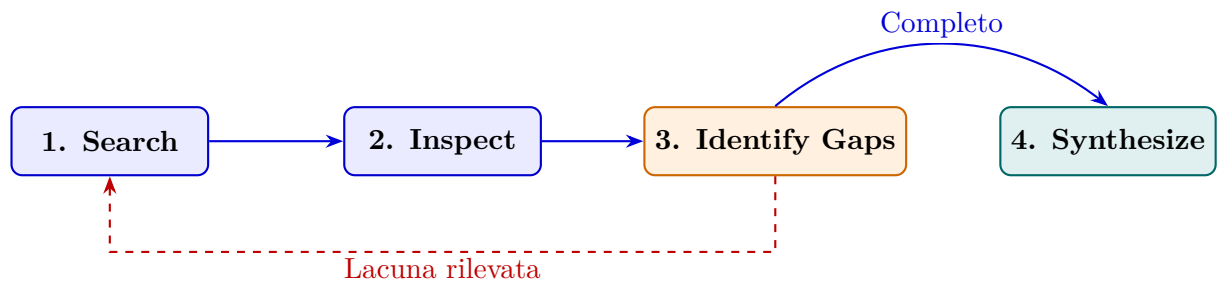


Figura 1.1: Ciclo a retroazione adattiva (*Adaptive Feedback Loop*) dell'Agentic Search.

1.1.2 Information Retrieval vs. Data Retrieval (Basi di Dati)

La Rilevanza: Il Cardine dell'IR

Mentre in una base di dati un record soddisfa o viola in modo booleano un predicato logico, nell'Information Retrieval il fulcro è la stima del grado di **rilevanza** rispetto al bisogno informativo dell'utente: soggettiva, situata nel contesto operativo ed evolutiva nel tempo.

Dimensione	Data Retrieval (Database SQL)	Information Retrieval
Struttura dati	Rigidamente schematizzata (tabelle, tipi, vincoli)	Non strutturata o semi-strutturata (testo libero, HTML)
Linguaggio di query Matching	Formale e deterministico (SQL) Booleano esatto	Naturale o a parole chiave, spesso ambiguo Parziale e probabilistico (gradi di rilevanza)
Ordinamento	Nessuno se non esplicito (ORDER BY)	Fondamentale, basato sulla stima di rilevanza
Tolleranza all'errore	Nulla	Elevata (refusi, sinonimia, polisemia)

Tabella 1.1: Confronto tra Data Retrieval tradizionale e Information Retrieval.

1.2 I Due Macro-Obiettivi: Efficacia ed Efficienza

Un sistema di reperimento su larga scala persegue simultaneamente due obiettivi spesso in tensione:

1. **Efficacia (*Effectiveness*):** quanto accuratamente il ranking soddisfa il bisogno informativo dell'utente.
2. **Efficienza (*Efficiency*):** costo computazionale, throughput e latenza di risposta, tipicamente nell'ordine dei 10–50 millisecondi per query su archivi di miliardi di documenti.

L'efficienza di un motore di ricerca moderno dipende dall'interazione con l'architettura hardware sottostante: pipelining e processori superscalari, gerarchia di memoria e località di accesso, vettorizzazione SIMD e calcolo parallelo/distribuito su cluster (*sharding*).

1.3 Architettura di un Sistema di Information Retrieval

Un sistema IR adotta una separazione netta tra la **Pipeline Offline** (indicizzazione e addestramento dei modelli di ranking) e la **Pipeline Online** (interrogazione a bassa latenza), come illustrato in Figura 1.2.

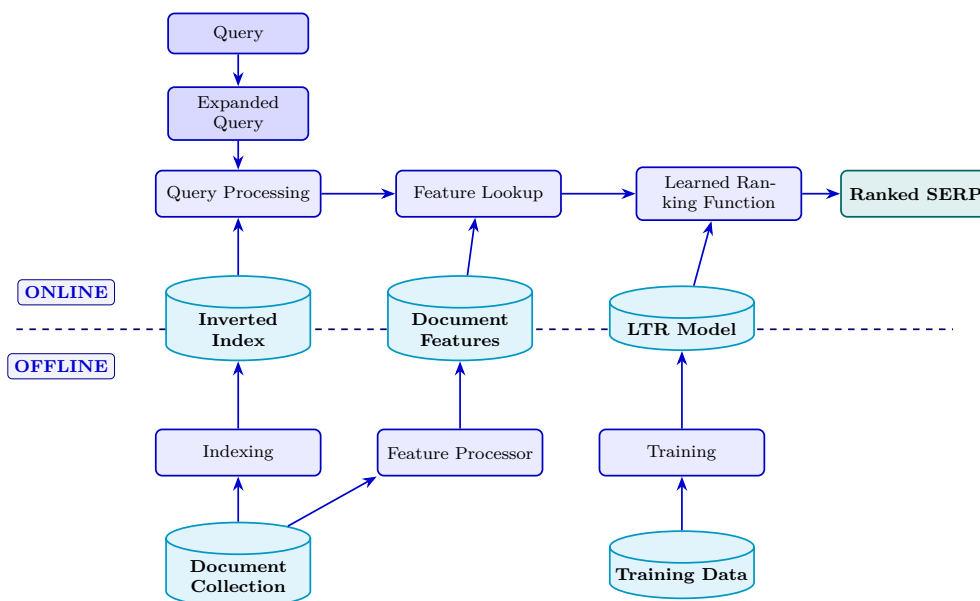


Figura 1.2: Architettura a griglia di un moderno motore di ricerca (pipeline Online vs. Offline).

1.3.1 Pipeline di Text Processing

Prima dell'indicizzazione, ogni flusso testuale attraversa: **tokenizzazione** (segmentazione in token lessicali), **normalizzazione/case folding** (minuscolo, diacritici), **filtraggio delle stop word** (rimozione di vocaboli ad alta frequenza ma privi di valore discriminante) e **stemming/lemmatizzazione** (riduzione di una forma flessa alla radice o al lemma canonico, es. "searching" → "search").

1.4 Proprietà del Linguaggio e Modelli di Reperimento

- **Legge di Zipf:** la frequenza $f(r)$ di un termine è inversamente proporzionale al suo rango r nella graduatoria di frequenza, $f(r) \propto 1/r$: poche parole frequentissime occupano la maggioranza delle occorrenze, mentre la maggior parte dei termini compare pochissime volte (*long tail*).
- **Legge di Heaps:** la dimensione del vocabolario cresce come legge di potenza rispetto al numero totale di parole N della collezione, $|V| = k \cdot N^\beta$ (con $0.4 \leq \beta \leq 0.6$): il vocabolario non si satura mai completamente.

I principali modelli di reperimento sono il **Modello Booleano** (match esatto tramite operatori logici, privo di punteggio di rilevanza), il **Vector Space Model** (query e documenti come vettori pesati con TF-IDF, rilevanza tramite similarità del coseno) e il **Modello Probabilistico BM25**, standard de facto del reperimento lessicale:

$$\text{score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d) \cdot (k_1 + 1)}{tf(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (1.1)$$

dove $\text{IDF}(t) = \ln\left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1\right)$ pesa la specificità del termine, k_1 (tipicamente 1.2–2.0) controlla la saturazione del term frequency e $b \in [0, 1]$ (tipicamente ≈ 0.75) normalizza la lunghezza del documento $|d|$ rispetto alla lunghezza media avgdl.

1.5 L’Inverted Index (Indice Invertito)

L’**Inverted Index** inverte l’orientamento logico della memorizzazione documentale, realizzando una mappatura $t \mapsto \mathcal{P}_t$, dove $\mathcal{P}_t$ è la lista ordinata dei documenti contenenti il termine t [1].

Definizione 1.5.1 (Inverted Index). Un Inverted Index è costituito da un **dizionario** (l’insieme ordinato dei termini unici, con frequenza documentale df_t e puntatore alla posting list) e, per ogni termine, da una **posting list**: un array ordinato per DocID crescente, i cui elementi possono includere metadati come frequenza locale (tf) e offset posizionali.

Esempio 1.5.1 (Collezione Giocattolo di Riferimento). Si consideri una piccola collezione di tre documenti sull’indicizzazione e la ricerca testuale. La Figura 1.3 illustra l’Inverted Index generato per alcuni termini chiave: il termine **a** compare nei documenti 1, 2, 3; **an** solo nel documento 1; **index** e **inverted** nei documenti 1, 2; **query** solo nel documento 3.

I costrutti booleani (AND, OR, NOT) e la ricerca per frase trovano diretta corrispondenza nell’interfaccia di ricerca avanzata dei motori commerciali (Tabella 1.2).

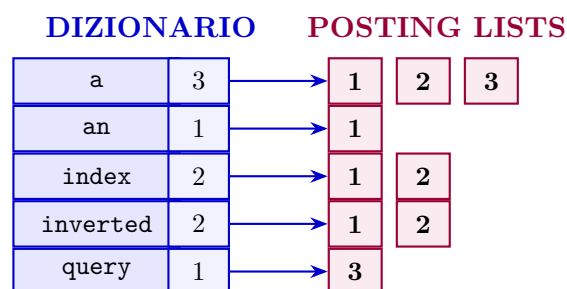


Figura 1.3: Struttura dell’Inverted Index: dizionario e posting list.

Operatore Logico IR	Campo Google Advanced Search	Sintassi
Congiunzione (AND)	<i>all these words</i>	tri-colour rat terrier
Ricerca per frase	<i>this exact word or phrase</i>	"rat terrier"
Disgiunzione (OR)	<i>any of these words</i>	miniature OR standard
Negazione (NOT)	<i>none of these words</i>	-rodent -"Jack Russell"
Intervallo numerico	<i>numbers ranging from...to</i>	10..35 kg

Tabella 1.2: Mappatura tra operatori booleani IR e Ricerca Avanzata di Google.

1.6 Tecniche di Compressione dell'Indice

La compressione dell'Inverted Index riduce l'occupazione di memoria (RAM/disco) convertendo un collo di bottiglia di I/O in un carico di decompressione CPU, generalmente molto più veloce.

1.6.1 Binary Interpolative Coding

Il **Binary Interpolative Coding** (Moffat e Stuiver [3]) comprime sequenze strettamente crescenti di interi (i DocID delle posting list) partizionandole ricorsivamente: data una sottolista di n interi nota per essere compresa in $[l, h]$, si codifica l'elemento centrale x_m (con $m = \lfloor (n+1)/2 \rfloor$) usando $\lceil \log_2 R \rceil$ bit, dove $R = h - l - n + 1$ è il numero di valori ammissibili, e si ricorre poi separatamente sulle due metà. Quando i DocID sono densamente clusterizzati, R tende a 1 e molti elementi vengono codificati a costo pressoché nullo.

1.6.2 Compressione Basata su Dizionario: LZ77

Per testi generali, l'algoritmo **Lempel-Ziv '77** (LZ77) comprime tramite una finestra scorrevole, sostituendo ripetizioni lessicali con triple (offset, length, next_char) riferite a occorrenze precedenti nella finestra. Questa famiglia di algoritmi è alla base di strumenti come gzip, 7-Zip e LZ4.

1.7 Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità

I candidati selezionati dall'Inverted Index vengono inoltrati a un modello di Machine Learning che li riordina (**Learning-to-Rank**, LTR) [4]. Il dataset di addestramento associa a ciascuna query una lista di documenti con giudizio di rilevanza graduato $y \in \{0, \dots, 4\}$ (da irrilevante a perfettamente rilevante); un modello calcola uno score $s_i = f_\theta(\vec{x}_{q,d_i})$ per ciascuna coppia query-documento, e la lista finale viene ottenuta ordinando per punteggio decrescente.

Il Cardine dell'LTR

The goal is to learn the ranking, not the label: l'obiettivo non è stimare accuratamente il singolo valore y , ma l'ordinamento relativo ottimale, massimizzando metriche cumulative come **NDCG** o **MAP**.

Poiché l'operazione di ordinamento è discreta e a gradini, il gradiente delle metriche di ranking rispetto ai parametri del modello è nullo quasi ovunque:

non è possibile applicare direttamente la discesa del gradiente. La letteratura ha sviluppato tre famiglie di soluzioni:

1. **Pointwise**: tratta ogni documento come un'istanza indipendente (regressione o classificazione su y), ignorando la natura comparativa del ranking.
2. **Pairwise** (es. RankNet): confronta coppie di documenti, modellando con una sigmoide differenziabile la probabilità che d_i preceda d_j .
3. **Listwise** (es. LambdaMART): ottimizza l'intera lista tramite gradienti surrogati λ_{ij} , pesati con la variazione effettiva di NDCG derivante dallo scambio di d_i e d_j ; combinato con alberi di decisione potenziati, è stato per oltre un decennio lo standard nei motori di ricerca commerciali.

1.8 Evoluzione verso il Neural Information Retrieval

Dal 2018, i modelli Transformer (BERT) hanno inaugurato l'era del Neural Information Retrieval [5, 6], pre-addestrati tramite **Masked Language Modeling**. Gli approcci neurali per il reperimento si dividono in due famiglie:

- **Interazione (Cross-Encoder)**: query e documento sono concatenati ed elaborati congiuntamente da un Transformer, catturando interazioni profonde a costo computazionale elevato ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$); usati solo come re-ranker su un piccolo insieme di candidati.
- **Rappresentazione (Bi-Encoder)**: query e documento sono proiettati indipendentemente in uno spazio vettoriale da due encoder distinti; i vettori dei documenti sono pre-calcolabili offline, riducendo il matching online a un prodotto scalare o a una ricerca di vicini approssimati.

Tra i modelli a rappresentazione si distinguono tre famiglie di embedding: **dense single-vector** (un unico vettore denso per documento/query, interrogato con indici k-ANN su grafi come HNSW), **dense multi-vector** (es. ColBERT [5], un vettore per token con aggregazione *Late Interaction*/MaxSim, $S(q, d) = \sum_{t_q \in q} \max_{t_d \in d} (\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T)$) e **sparse single-vector** (es. SPLADE [6], interpretabile per design e più efficiente in spazio/tempo rispetto al multi-vector).

k -Approximate Nearest Neighbor Search

Su collezioni di milioni di documenti rappresentati come vettori densi ad alta dimensionalità, il recupero esatto dei k più simili alla query è computazionalmente proibitivo: si ricorre quindi a indici di **ricerca approssimata dei vicini** (k -ANN), che scambiano una minima perdita di accuratezza per un guadagno di ordini di grandezza in velocità. **TusKANNy** (<https://tuskanny.com/>), sviluppato da ISTI-CNR e Università di Pisa, ne è un esempio open-source ad alte prestazioni scritto in Rust.

Capitolo 2

Valutazione dei Sistemi di Information Retrieval

2.1 Architettura di un Motore di Ricerca

Prima di affrontare la valutazione, è utile inquadrare l'architettura complessiva di un moderno motore di ricerca, che può essere schematizzata nel seguente diagramma a blocchi:

I componenti principali sono:

- **Web Crawler (Document Collection):** Un agente automatico che percorre il Web seguendo i collegamenti ipertestuali e scarica i documenti da indicizzare. La collezione risultante è la base di lavoro del sistema.
- **Indexing:** La pipeline di pre-elaborazione che trasforma i documenti grezzi in una rappresentazione indicizzabile: parsing HTML, tokenizzazione, normalizzazione, rimozione delle stopwords e stemming/lemmatizzazione.
- **Inverted Index:** La struttura dati centrale del motore di ricerca: per ogni termine mappa la lista dei documenti che lo contengono (posting list), consentendo un recupero efficiente.
- **Query Processing e Query Expansion:** Il modulo che analizza la query dell'utente, la normalizza e — facoltativamente — la espande con sinonimi o termini correlati per aumentare il richiamo.
- **Feature Extraction:** Estrazione di segnali numerici per ciascuna coppia query-documento (TF-IDF, BM25, PageRank, freschezza, segnali di click, ecc.).

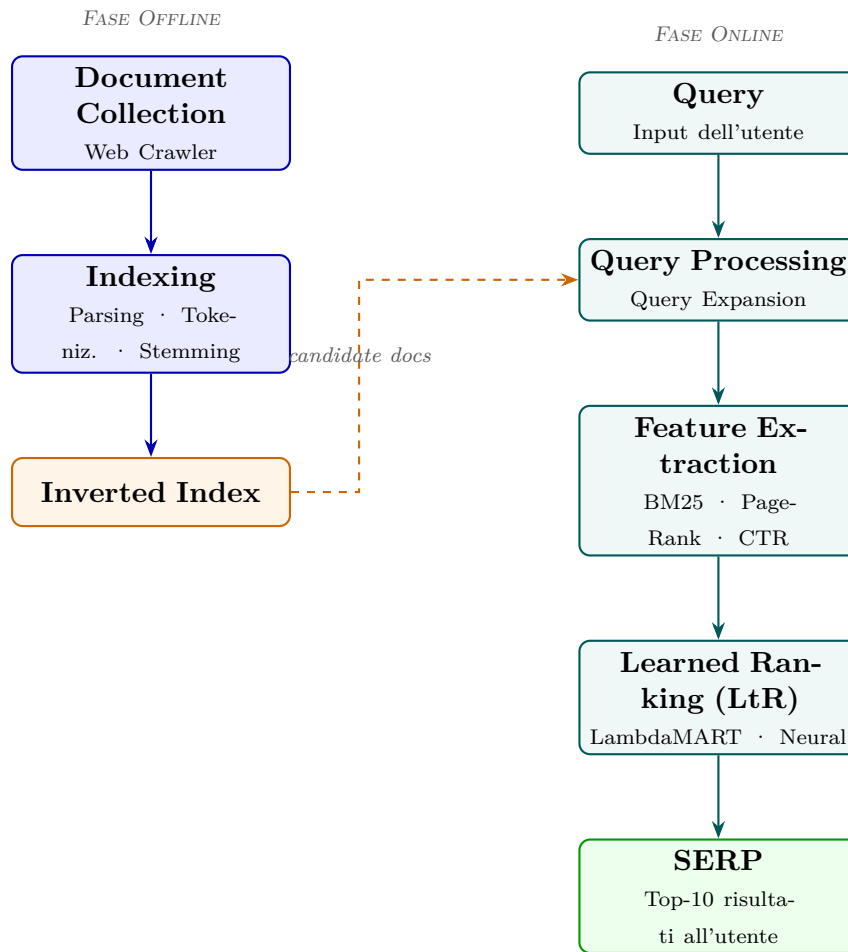


Figura 2.1: Diagramma a blocchi dell'architettura di un motore di ricerca moderno: pipeline offline (indicizzazione) e pipeline online (elaborazione query e ranking).

- **Learned Ranking Function (Learning to Rank):** Un modello statistico o di apprendimento automatico (ad es. LambdaMART, LambdaRank) addestrato a ordinare i documenti massimizzando una metrica di qualità, tipicamente NDCG.
- **SERP (Search Engine Results Page):** La pagina dei risultati presentata all'utente, comprensiva di titolo, snippet, URL e, nei motori moderni, risposte generate (Featured Snippets, AI Overviews).

2.2 Come Valutare la Qualità di un Motore di Ricerca

Un sistema di Information Retrieval è un artefatto complesso che integra decine di componenti eterogenei: come si valuta qualcosa di così articolato? La risposta della comunità scientifica distingue nettamente due dimensioni complementari:

1. Metriche di Efficienza:

- *Velocità di indicizzazione:* Quanti documenti per ora il sistema è in grado di processare e inserire nell'indice.
- *Indicizzazione incrementale:* Capacità di aggiornare l'indice con nuovi flussi documentali (ad es. +10 000 prodotti al giorno in un e-commerce) senza interruzioni di servizio.
- *Velocità di ricerca (Latenza e Throughput):* Tempo di risposta (latenza al 95° o 99° percentile) e consumo di CPU/memoria nell'elaborazione di migliaia di query al secondo su collezioni di svariati milioni o miliardi di documenti.

2. Metriche di Efficacia e Qualità dei Servizi Accessori:

- Qualità delle raccomandazioni correlate, accuratezza della correzione ortografica delle query (*spell checking*) e chiarezza dei riassunti testuali visualizzati (*snippet generation*).
- **Efficacia intrinseca della graduatoria:** Misura la capacità fondamentale del motore di ricerca di soddisfare l'utente, ordinando i documenti pertinenti nelle prime posizioni della pagina dei risultati (*Search Engine Results Page*, SERP).

2.3 Misurare la Soddisfazione dell'Utente

Nel contesto operativo dei motori di ricerca commerciali, l'obiettivo ultimo è massimizzare la soddisfazione dell'utente (*User Happiness*). Poiché la felicità è uno stato cognitivo non osservabile direttamente, i sistemi di produzione ricorrono a proxy comportamentali online:

- **Clic e Click-Through Rate (CTR):** Il numero e la proporzione di clic ricevuti dai risultati restituiti.
 - *Criticità:* Titoli sensazionalistici o riassunti ingannevoli (*clickbait*) inducono clic elevati senza che il documento soddisfi realmente l'utente.
 - *La ricerca a zero clic (Zero-Click Search):* L'assenza di clic non è necessariamente un segnale negativo. Quando un utente cerca informazioni fattuali (ad es. il meteo, l'orario di un volo, una conversione di valuta o una definizione enciclopedica), la risposta visualizzata direttamente tramite *Featured Snippet* o *Knowledge Graph* soddisfa istantaneamente il bisogno senza richiedere alcun clic.
- **Conversioni e Transazioni Economiche:** Negli e-commerce, la quota di sessioni di ricerca che sfociano nell'acquisto di un prodotto e l'ammontare monetario speso dall'utente costituiscono metriche primarie di business.
- **Fidelizzazione e Visite Ripetute:** La percentuale di utenti che tornano ad adoperare il motore di ricerca entro un intervallo temporale predefinito (una settimana, un mese).
- **Dwell Time (Tempo di Permanenza):** L'intervallo di tempo che intercorre tra il momento in cui l'utente clicca su un risultato della SERP e l'istante in cui fa ritorno alla pagina dei risultati (*bouncing back*). Un Dwell Time prolungato indica generalmente che il documento ha catturato l'attenzione ed è risultato informativo, mentre un ritorno immediato alla SERP denota insoddisfazione.

La Valutazione nell'Era dei Motori Generativi e LLM

L'emergere di paradigmi basati su modelli linguistici (come ChatGPT, SearchGPT e Perplexity) ridefinisce radicalmente il concetto di Dwell Time e CTR: l'utente non naviga più attraverso link esterni, ma consuma sintesi testuali generate direttamente. In tale scenario, l'accuratezza fattuale, l'assenza di allucinazioni e l'attribuzione corretta delle fonti rimpiazzano il conteggio dei clic tradizionali.

Il dibattito accademico su come valutare questi sistemi è aperto e vivace. Alcune posizioni emerse in aula:

- **Numero di turni conversazionali come proxy di fallimento (Dino):** In un sistema conversazionale, un elevato numero di turni necessari per soddisfare un bisogno informativo può segnalare che il sistema non ha compreso correttamente l'intento dell'utente fin dal primo scambio. Il Dwell Time classico perde di significato; si ragiona invece sulla lunghezza della conversazione.
- **Varianza dovuta al prompting (Gabriele):** I modelli generativi sono sensibili alla formulazione della richiesta: prompt leggermente diversi per lo stesso bisogno informativo possono produrre risposte qualitativamente molto differenti. Questa varianza intrinseca rende difficile una valutazione riproducibile.
- **Differenza ontologica IR classico vs. LLM generativo (Bruno):** Un motore classico *recupera* documenti esistenti; un LLM generativo *sintetizza* una risposta nuova. Si tratta di due paradigmi ontologicamente distinti: nel primo caso si può misurare la rilevanza rispetto a un corpus noto; nel secondo, l'output è unico e non confrontabile con un gold standard preesistente.
- **Il paradigma RAG (Retrieval-Augmented Generation):** Una risposta parziale alla critica delle allucinazioni è l'architettura RAG, che combina un motore di recupero tradizionale con un LLM generativo: si recuperano prima i documenti rilevanti dall'indice, poi si condiziona la generazione su di essi. Questo approccio riduce le allucinazioni e permette di citare le fonti, riavvicinando il sistema generativo al paradigma di valutazione classico.

2.4 Il Paradigma di Cranfield e le Collezioni Benchmark

Sebbene i segnali online forniscano un feedback prezioso, essi risultano rumorosi, soggetti a forti bias di posizionamento (*Position Bias*) e non riproducibili sperimentalmente prima del rilascio in produzione. Per consentire valutazioni comparative controllate, la ricerca in IR adotta il **Paradigma di Cranfield**, sviluppato originariamente da Cyril Cleverdon negli anni '60 [7].

È importante contestualizzare il momento storico: Cleverdon lavorava su **biblioteche fisiche**, con schedari cartacei e cataloghi aeronautici. Il suo esperimento — condotto prima dell'avvento dei calcolatori elettronici diffusi — consisteva nel valutare manualmente quanto bene diversi sistemi di classificazione documentale riuscissero a soddisfare bisogni informativi reali. Eppure, il framework metodologico che ne derivò è rimasto il fondamento della valutazione in IR per oltre sessant'anni.

Definizione 2.4.1 (Collezione di Test (Test Collection)). Una collezione di test standard per l'Information Retrieval è una tripletta formale $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ costituita da:

1. Una collezione documentale benchmark $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$;
2. Una suite benchmark di bisogni informativi e query $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$;
3. Un insieme di giudizi di rilevanza (*Relevance Judgments* o *grels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, che associa a ciascuna coppia query-documento un valore di rilevanza (binario o graduato).

2.4.1 La Sfida della Scalabilità Combinatoria

La realizzazione manuale esaustiva dei giudizi di rilevanza è materialmente impossibile per archivi realistici. Si consideri uno scenario benchmark con:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documenti}, \quad |\mathcal{Q}| = 50\,000 \text{ query}$$

Lo spazio complessivo delle coppie ammonta a:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 5 \times 10^4 = 2.5 \times 10^{11} \text{ coppie query-documento}$$

Ipotizzando che un annotatore umano impieghi in media 2.5 secondi per valutare la rilevanza di un documento:

$$\begin{aligned} \text{Tempo Totale} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ secondi} \\ &\approx 173\,611\,111 \text{ ore/persona} \end{aligned}$$

Un impegno titanico che richiederebbe decine di migliaia di anni di lavoro umano ininterrotto! Per ovviare a ciò, i benchmark limitano i giudizi a un sottoinsieme accuratamente campionato di documenti.

2.4.2 Origini delle Query di Test

Una questione metodologica fondamentale riguarda la provenienza delle query che compongono il benchmark $\mathcal{Q}$:

- **Motori Web commerciali — Query Logs:** I grandi motori di ricerca (Google, Microsoft Bing) campionano le query di test direttamente dai propri *query log*, cioè dai log di ricerca reali degli utenti. Dataset pubblici storici come l'AOL Query Log (2006) e i dataset Microsoft ORCAS/MIMICS contengono milioni di query reali. Questo approccio garantisce che le query riflettano bisogni informativi autentici con la distribuzione di frequenza reale (incluse le query rare della coda lunga).
- **Scenario classico — Esperti a tavolino:** Nelle prime collezioni (Cranfield, TREC), esperti di dominio stilavano manualmente liste di *information needs* plausibili, simulando i bisogni di un ipotetico utente. Il processo era deliberato e supervisionato, con attenzione a coprire diversi tipi di bisogno informativo (noto, esplorativo, transazionale).
- **Perché non generare query casuali dai documenti:** Una strategia apparentemente ovvia — campionare termini o frasi dai documenti della collezione per costruire query sintetiche — è metodologicamente scorretta. Le query così generate non rifletterebero gli intenti reali degli utenti, ma solo il vocabolario dei documenti, producendo una valutazione distorta e priva di validità ecologica.

2.4.3 Crowdsourcing: Opportunità e Limiti

Una strategia economica consiste nel delegare la valutazione a lavoratori retribuiti su piattaforme online di crowdsourcing (come Amazon Mechanical Turk). Tuttavia, la letteratura scientifica evidenzia come tali giudizi siano affetti da:

- Elevatissima varianza statistica;
- Minore competenza e presenza di spammer;
- Necessità di filtri rigorosi di consenso (ad es. votazione di maggioranza o calcolo dell'accordo inter-annotatore tramite coefficiente κ di Cohen).

Una strategia più recente e sempre più diffusa è l'impiego di **LLM come Valutatori** (*LLM-as-a-Judge*):

- Modelli come GPT-4 vengono utilizzati con prompt strutturati per valutare la rilevanza di coppie query-documento, in sostituzione o affiancamento agli annotatori umani. La scalabilità economica è enormemente superiore al crowdsourcing tradizionale: il costo per giudizio scende di uno o due ordini di grandezza.
- Tuttavia, questa metodologia presenta **bias intrinseci** documentati: preferenza per testi più lunghi (indipendentemente dalla qualità), *self-preference bias* (i modelli tendono a valutare meglio testi simili ai propri output di addestramento) e sensibilità all'ordine di presentazione dei candidati. È quindi necessario progettare con cura i prompt di valutazione e condurre studi di calibrazione rispetto ai giudizi umani.

2.4.4 L'Infrastruttura TREC (*Text REtrieval Conference*)

Promossa dal National Institute of Standards and Technology (NIST), la conferenza **TREC** rappresenta il punto di riferimento internazionale per la valutazione delle tecnologie di recupero dell'informazione. TREC è strutturata in tracce tematiche (*Tracks*), ciascuna incentrata su specifici casi d'uso (ricerca Web, ricerca clinica, domande-risposte, deep retrieval). Il risultato prodotto da un sistema software che esegue un compito su una collezione di test prende il nome di **Run**.

La Tabella 2.1 documenta l'evoluzione dimensionale delle collezioni benchmark storiche utilizzate nella ricerca in Information Retrieval [1], evidenziando il salto di scala che ha reso indispensabile la nascita di TREC.

Come si evince dai dati, nelle prime collezioni degli anni '60 e '70 (come Cranfield e Medline), le ridotte dimensioni (poche migliaia di record e 1–2 MB) consentivano un'annotazione esaustiva manuale. Con l'avvento di collezioni realistiche da centinaia di migliaia di documenti e gigabyte di testo come TREC, lo spazio combinatorio delle valutazioni è esploso, imponendo l'adozione di metodologie di campionamento selettivo.

2.4.5 La Tecnica del Pooling

Per campionare i documenti da sottoporre ad annotazione umana senza dover valutare l'intera collezione, nei benchmark TREC si adotta la tecnica del **Pooling** (Figura 2.2):

Collezione	Documenti (N)	Query	Dimensione (MB)	Termini/Doc	Giudizi (Q-D)
ADI	82	35	—	—	—
AIT	2 109	14	2	400.0	> 10 000
CACM	3 204	64	2	24.5	—
CISI	1 460	112	2	46.5	—
Cranfield	1 400	225	2	53.1	Completi
LISA	5 872	35	3	—	—
Medline	1 033	30	1	—	—
NPL	11 429	93	3	—	—
OSHMED	348 566	106	400	250.0	16 140
Reuters	21 578	672	28	131.0	—
TREC (Tipica)	740 000	200	2 000 (2 GB)	89–3 543	> 100 000

Tabella 2.1: Panoramica comparativa dei principali corpora benchmark storici di valutazione.

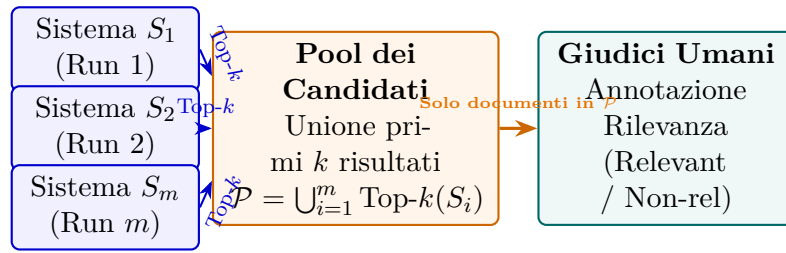


Figura 2.2: Processo di Pooling per la selezione controllata dei documenti da annotare.

1. Si raccolgono le risposte fornite da m sistemi di recupero eterogenei per una determinata query.
2. Si estraggono i primi k documenti restituiti da ciascun sistema (tipicamente $k = 50$ o $k = 100$).
3. Si calcola l'**unione** di tali insiemi eliminando i duplicati, formando il pool $\mathcal{P}$.
4. Gli annotatori umani valutano la rilevanza unicamente per i documenti appartenenti a $\mathcal{P}$.

L'Assunzione Fondamentale del Pooling

Tutti i documenti dell'archivio che non compaiono nel pool (*unpooled*) vengono assunti convenzionalmente come **non rilevanti** ($r = 0$). Sebbene tale assunzione introduca un potenziale bias a sfavore di algoritmi futuri con paradigmi radicalmente differenti, studi empirici condotti da NIST confermano che, a fronte di un pool formato da sistemi sufficientemente diversificati, le conclusioni comparative tra motori di ricerca rimangono stabili e affidabili.

2.5 Bisogno Informativo vs. Query

Una distinzione concettuale decisiva riguarda il divario tra bisogno informativo e formulazione della query:

- **Bisogno Informativo (*Information Need*)**: Rappresenta lo stato cognitivo e l'obiettivo reale dell'utente. Ad esempio: *"Il fondo della mia piscina sta diventando nero e necessita di essere ripulito"*.
- **Query**: L'espressione testuale sintetica digitata nel box di ricerca. Ad esempio: "pulitore piscina".

Proprietà 2.5.1 (Principio di Valutazione della Rilevanza). *La rilevanza di un documento deve essere sempre stimata rispetto all'**Information Need** sottostante, mai rispetto alla mera sequenza di caratteri che compone la query.*

2.6 Metriche di Valutazione Non Ordinate (*Unranked Measures*)

Nel contesto del reperimento non ordinato (come nel modello booleano classico), il sistema restituisce un insieme non graduato di documenti. La va-

lutazione si fonda sulla suddivisione dell'universo documentale mediante la matrice di contingenza (Tabella 2.2):

	Documento Rilevante	Documento Non Rilevante
Documento Reperito	True Positive (TP)	False Positive (FP)
Documento Non Reperito	False Negative (FN)	True Negative (TN)

Tabella 2.2: Matrice di contingenza per la classificazione binaria query-documento.

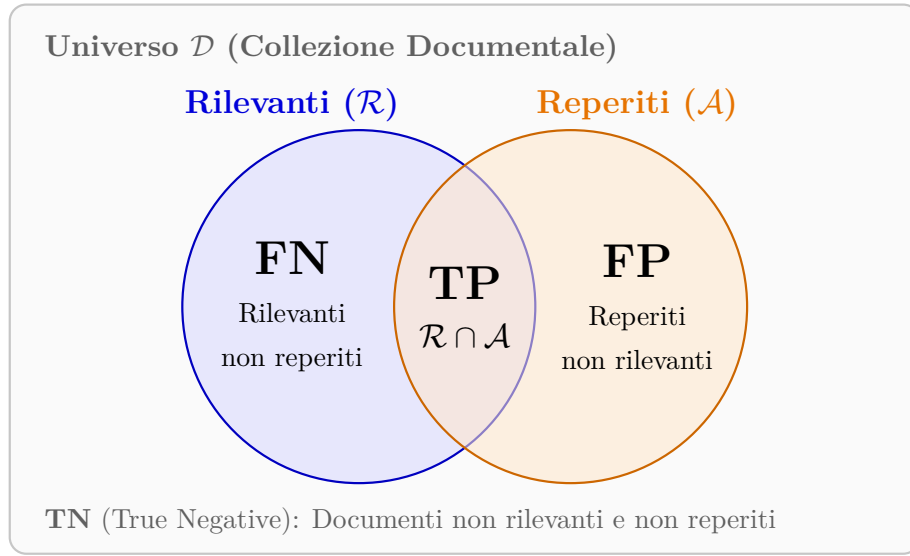


Figura 2.3: Rappresentazione insiemistica del reperimento documentale e delle regioni della matrice di confusione.

2.6.1 Precisione, Richiamo e loro Trade-off

Definizione 2.6.1 (Precisione ($Precision$, P)). La frazione dei documenti restituiti dal sistema che risultano effettivamente rilevanti:

$$P = \frac{TP}{TP + FP} = P(\text{rilevante} \mid \text{reperito}) \quad (2.1)$$

Definizione 2.6.2 (Richiamo ($Recall$, R)). La frazione dei documenti rilevanti complessivamente presenti nella collezione che il sistema è riuscito a reperire:

$$R = \frac{TP}{TP + FN} = P(\text{reperito} \mid \text{rilevante}) \quad (2.2)$$

Esiste un intrinseco **trade-off** tra Precisione e Richiamo:

- Un sistema che restituisce l'intera collezione documentale (Retrieved = $\mathcal{D}$) ottiene banalmente un richiamo perfetto $R = 100\%$, ma la sua precisione crolla a livelli minimi ($P \approx 0$).
- Viceversa, un sistema conservativo che restituisce unicamente il singolo documento più sicuro può conseguire $P = 100\%$, pagando il prezzo di un richiamo trascurabile.

2.6.2 La Media Armonica: F-Measure (F_1 -Score)

Per sintetizzare Precisione e Richiamo in un unico valore scalare che premi l'equilibrio tra le due grandezze, si adotta la **F-Measure** (o F_1 -score), definita come la **media armonica** di P ed R :

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (2.3)$$

Proprietà 2.6.1 (Proprietà della Media Armonica). *La media armonica gode di proprietà matematiche fondamentali per la valutazione:*

1. È sempre minore o uguale alla media geometrica e alla media aritmetica:

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a + b}{2}$$

2. Quando i due argomenti differiscono notevolmente, la media armonica crolla verso il **valore minimo** anziché posizionarsi a metà strada.

Esempio 2.6.1 (Comportamento Limite dell'F-Measure). Si consideri un sistema banale che restituisce l'intera collezione: $R = 1.0$ e $P = 0.0001$.

- La media aritmetica assegna un punteggio fuorviante pari a:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- La media armonica (F_1) demolisce giustamente la valutazione:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

Ciò dimostra per quale motivo la media aritmetica sia inadeguata e l'F-measure costituisca la scelta corretta.

2.6.3 La Misura F_β Pesata

Qualora si desideri attribuire pesi differenziati a Precisione e Richiamo, si introduce un parametro di ponderazione $\alpha \in [0, 1]$:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad (2.4)$$

Esprimendo α in funzione di un parametro β tale che $\alpha = \frac{1}{1+\beta^2}$, si ricava la formulazione classica di F_β :

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \quad (2.5)$$

- Valori di $\beta < 1$ (ad es. $\beta = 0.5$) enfatizzano la **Precisione** rispetto al Richiamo (utile nelle ricerche su Web generico).
- Valori di $\beta > 1$ (ad es. $\beta = 2.0$) enfatizzano il **Richiamo** rispetto alla Precisione (fondamentale nelle ricerche brevettuali, legali e mediche, dove omettere un documento rilevante comporta danni gravi).

2.7 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria)

Nei motori di ricerca moderni i documenti non vengono restituiti come insiemi informi, ma ordinati in base a uno score di rilevanza decrescente. Inoltre, gli utenti Web raramente oltrepassano la prima pagina dei risultati (i primi 10 o 20 elementi).

2.7.1 Precision@k ($P@k$) e Recall@k ($R@k$)

Definizione 2.7.1 (Precision@k). Fissata una soglia di rango $k \in \mathbb{N}^+$, la Precision@k corrisponde alla proporzione di documenti rilevanti presenti nei primi k posti della graduatoria:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i \quad (2.6)$$

dove $\text{rel}_i \in \{0, 1\}$ denota la rilevanza binaria del documento al rango i .

Esempio 2.7.1 (Calcolo di Precision@k). Si consideri una lista ordinata in cui la sequenza di rilevanza binaria dei primi 5 documenti è data da $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (si noti che $P@4 < P@3$)
- $P@5 = \frac{2}{5} = 0.40$

La Distorsione di $P@k$ nella Media tra Query

Sebbene $P@k$ sia intuitivo per stimare la qualità della prima pagina, presenta un difetto metodologico critico quando viene mediato su insiemi di query eterogenee:

- Se per una query specifica q_A esistono solo $R_A = 2$ documenti rilevanti nell'intera collezione, il valore massimo teoricamente raggiungibile per $P@10(q_A)$ è pari a $\frac{2}{10} = 0.20$, anche qualora il motore collochi perfettamente entrambi i documenti ai primi due ranghi!
- Calcolando la media aritmetica di $P@10$ tra query, il motore viene ingiustamente penalizzato dalla rarità intrinseca dei documenti rilevanti di alcune query, e non da un errore dell'algoritmo di ranking.

In modo analogo, la metrica complementare $Recall@k$ ($R@k$) calcola la frazione dei documenti rilevanti complessivi reperiti entro il rango k :

$$Recall@k = \frac{1}{R} \sum_{i=1}^k rel_i \quad (2.7)$$

2.7.2 Average Precision (AP)

L'**Average Precision** (AP) sintetizza l'intera curva Precision-Recall in un singolo indicatore scalare per una specifica query, valutando la precisione istantanea in corrispondenza di ogni posizione in cui il richiamo aumenta.

Definizione 2.7.2 (Average Precision). Dato un ordinamento di N documenti per una query con R documenti rilevanti totali presenti nella collezione, l'AP è definita come:

$$AP = \frac{1}{R} \sum_{k=1}^N P@k \cdot rel_k \quad (2.8)$$

Proprietà 2.7.1 (Interpretazione Geometrica e Ruolo di $1/R$). *L’Average Precision possiede una chiara interpretazione analitica:*

1. Ogni volta che si incontra un documento rilevante al rango k , il richiamo compie un gradino discreto di ampiezza $\Delta\text{Recall} = \frac{1}{R}$. L’AP calcola la somma delle aree dei rettangoli $P@k \cdot \Delta\text{Recall}$, rappresentando l’**approssimazione numerica dell’Area Sotto la Curva Precision-Recall (AUC-PR)**.
2. La normalizzazione per R (il numero totale di documenti rilevanti nella collezione) garantisce che i documenti rilevanti non recuperati entro i primi N ranghi contribuiscano implicitamente con precisione pari a 0, penalizzando severamente i sistemi affetti da scarso richiamo.

	Ordinamento del Motore di Ricerca →				
Posizione (Rango):	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
Documento:	Rel	NonRel	Rel	NonRel	Rel
Precision@k:	1.00	—	0.67	—	0.60

Figura 2.4: Rappresentazione schematica del calcolo di Average Precision su 5 documenti con $R = 3$ rilevanti totali.

Esempio 2.7.2 (Calcolo di Base con $R = 3$ Rilevanti). Si consideri lo scenario di Figura 2.4 con $R = 3$ documenti rilevanti totali, posizionati ai ranghi $k_1 = 1$, $k_2 = 3$ e $k_3 = 5$:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

L’Average Precision risulta:

$$AP = \frac{1}{3} \left(1.00 + \frac{2}{3} + \frac{3}{5} \right) = \frac{1}{3} (2.267) \approx 0.756 \quad (75.6\%) \quad (2.9)$$

Esempio 2.7.3 (Confronto Comparativo tra Due Ordinamenti). Si consideri una query per la quale esistono $R = 6$ documenti rilevanti nella collezione. Due motori di ricerca alternativi restituiscono i seguenti primi 10 documenti:

- **Ranking #1:** Rilevanti concentrati nelle prime posizioni:

⟨Rel, NonRel, Rel, Rel, Rel, Rel, NonRel, NonRel, NonRel, Rel⟩

Rango (k)	Ranking #1			Ranking #2		
	Esito	Recall	Precision	Esito	Recall	Precision
1	Rel	0.17	1.00	NonRel	0.00	—
2	NonRel	0.17	—	Rel	0.17	0.50
3	Rel	0.33	0.67	NonRel	0.17	—
4	Rel	0.50	0.75	NonRel	0.17	—
5	Rel	0.67	0.80	Rel	0.33	0.40
6	Rel	0.83	0.83	Rel	0.50	0.50
7	NonRel	0.83	—	Rel	0.67	0.57
8	NonRel	0.83	—	NonRel	0.67	—
9	NonRel	0.83	—	Rel	0.83	0.56
10	Rel	1.00	0.60	Rel	1.00	0.60
Average Precision (AP)			0.78	0.52		

Tabella 2.3: Calcolo comparativo dettagliato di Average Precision tra Ranking #1 e Ranking #2.

- **Ranking #2:** Rilevanti dispersi e ritardati:

⟨NonRel, Rel, NonRel, NonRel, Rel, Rel, Rel, NonRel, Rel, Rel⟩

La Tabella 2.3 traccia l'evoluzione rango per rango di Precisione e Richiamo per entrambi i sistemi.

I rispettivi valori di AP risultano:

$$AP(\text{Ranking \#1}) = \frac{1.00 + 0.67 + 0.75 + 0.80 + 0.83 + 0.60}{6} = \frac{4.65}{6} \approx \mathbf{0.78}$$

$$AP(\text{Ranking \#2}) = \frac{0.50 + 0.40 + 0.50 + 0.57 + 0.56 + 0.60}{6} = \frac{3.13}{6} \approx \mathbf{0.52}$$

Sebbene entrambi i motori raggiungano un richiamo finale identico ($R = 100\%$ sui 10 documenti), Ranking #1 ottiene un punteggio nettamente superiore (0.78 vs 0.52) poiché premia la densità di documenti rilevanti nella parte alta della graduatoria.

2.7.3 Mean Average Precision (MAP)

Quando si valuta un motore di ricerca su un banco di prova comprendente $|Q|$ bisogni informativi distinti, si calcola la **Mean Average Precision** (MAP) come media aritmetica dei valori di AP:

Definizione 2.7.3 (Mean Average Precision).

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q) \quad (2.10)$$

Proprietà 2.7.2 (Macro-Averaging vs. Micro-Averaging). *Il MAP adotta formalmente una procedura di **Macro-Averaging**: ciascuna query possiede un peso identico pari a $1/|\mathcal{Q}|$.*

- *Se si adottasse il Micro-Averaging (ovvero sommare tutte le precisioni istantanee e dividere per il numero totale di documenti rilevanti dell'intera collezione), la misura verrebbe totalmente monopolizzata da pochissime query iper-generiche (ad es. "film" o "notizie", aventi decine di migliaia di documenti rilevanti), mascherando il fallimento del motore sulle query rare e specifiche (*long-tail*).*
- *Il Macro-Averaging garantisce che ogni singola esigenza informativa dell'utente riceva pari dignità nella valutazione finale.*

Esempio 2.7.4 (Calcolo di MAP su Due Query). Si consideri la valutazione congiunta di due query q_1 e q_2 :

- **Query 1 ($R_1 = 5$ rilevanti totali):** Il sistema restituisce 5 rilevanti ai ranghi $k \in \{1, 3, 6, 9, 10\}$ con precisioni istantanee pari a 1.00, 0.67, 0.50, 0.44 e 0.50:

$$AP(q_1) = \frac{1.00 + 0.67 + 0.50 + 0.44 + 0.50}{5} = \frac{3.11}{5} \approx \mathbf{0.62}$$

- **Query 2 ($R_2 = 3$ rilevanti totali):** Il sistema restituisce i 3 rilevanti ai ranghi $k \in \{2, 5, 7\}$ con precisioni istantanee pari a 0.50, 0.40 e 0.43:

$$AP(q_2) = \frac{0.50 + 0.40 + 0.43}{3} = \frac{1.33}{3} \approx \mathbf{0.44}$$

Il valore di Mean Average Precision del motore sui due compiti ammonta a:

$$MAP = \frac{AP(q_1) + AP(q_2)}{2} = \frac{0.62 + 0.44}{2} = \mathbf{0.53} \quad (2.11)$$

2.7.4 Metriche Troncate per il Web e Recommender Systems: AP@k e MAP@k

Nei moderni motori di ricerca su Web e nei sistemi di raccomandazione, calcolare l'AP sull'intera collezione risulta impraticabile (perché il numero totale R di rilevanti non è noto a priori) e poco rilevante per l'esperienza utente. Si adotta pertanto la variante troncata a soglia fissa K (**MAP@K**):

Definizione 2.7.4 (AP@K e MAP@K). Fissata la soglia di taglio K (tipicamente $K = 10$ o $K = 20$), l’Average Precision at K per la query q_i è definita come:

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^K P@k(q_i) \cdot \text{rel}(q_i, k) \quad (2.12)$$

dove il fattore di normalizzazione K_i è dato dal numero di documenti rilevanti presenti nei primi K posti:

$$K_i = \sum_{k=1}^K \text{rel}(q_i, k) \quad (\text{oppure } K_i = \min(R_i, K)) \quad (2.13)$$

La metrica globale **MAP@K** su un insieme di query $\mathcal{Q}$ corrisponde alla media:

$$MAP@K(\mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} AP@K(q_i) \quad (2.14)$$

Questa formulazione risolve brillantemente il limite di $P@K$: normalizzando per K_i anziché rigidamente per K , una query con soli 2 rilevanti che li colloca ai primi 2 posti ottiene $AP@10 = 1.0$ anziché essere artificialmente limitata a 0.20.

2.7.5 Mean Reciprocal Rank (MRR)

In scenari quali le ricerche di elementi noti (*Known-Item Search*), le query navigazionali (ad es. cercare l’home page di un ateneo) o il reperimento di risposte fattuali, l’utente è interessato unicamente a individuare la **prima risposta corretta**. Lo sforzo cognitivo e il tempo speso dall’utente sono inversamente proporzionali alla posizione occupata dal primo risultato rilevante:

Definizione 2.7.5 (Reciprocal Rank e MRR). Sia $\text{rank}_1(q)$ la posizione del primo documento rilevante restituito per la query q . Il **Reciprocal Rank** (RR) è dato da:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \quad (2.15)$$

Il **Mean Reciprocal Rank** (MRR) è la media aritmetica su tutte le query:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \quad (2.16)$$

Se per una query il primo documento rilevante si trova al rango 1, il punteggio è 1.0; se scende al rango 2 diviene 0.5; se scende al rango 10 crolla a 0.1.

2.8 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG

Nelle applicazioni Web reali, l'ipotesi che un documento sia puramente rilevante (1) o non rilevante (0) risulta insufficiente. Si consideri la seguente query reale:

Query: "information retrieval unipi"

Una tipica SERP commerciale restituisce risultati con livelli qualitativi marcatamente differenti:

1. **DidaWiki del Corso (Docente, CFU, Programma, Materiale didattico):** Rappresenta la pagina principale che soddisfa pienamente l'intento dell'utente $\implies$ **Altamente Rilevante** ($r = 3$).
2. **Portale Ufficiale UniPi Valutami (Scheda d'esame e requisiti istituzionali):** Pagina autorevole ma meno ricca di dispense didattiche $\implies$ **Rilevante** ($r = 2$).
3. **Brochure PDF del Corso di Laurea Magistrale:** Contiene solo una breve menzione del corso $\implies$ **Marginalmente Rilevante** ($r = 1$).
4. **Articoli di cronaca o pagine non correlate:** $\implies$ **Irrilevante** ($r = 0$).

Tutti i primi tre documenti sono in qualche misura attinenti, ma un utente sarà soddisfatto solo se la pagina più informativa compare in cima. Per catturare questa granularità, la letteratura adotta scale a più livelli (tipicamente $r \in \{0, 1, 2, 3\}$ o $\{0, \dots, 4\}$).

L'approccio del **Discounted Cumulative Gain** (DCG), introdotto da Järvelin e Kekäläinen [8], poggia su due assunzioni fondamentali:

1. I documenti altamente rilevanti apportano un'utilità (*gain*) di gran lunga superiore rispetto a documenti solo marginalmente rilevanti.
2. Minore è la posizione ordinale (rango) di un documento nella graduatoria, minore è la probabilità che l'utente lo esamini; pertanto, il guadagno deve essere progressivamente scontato (*discounted*) verso il basso.

2.8.1 Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)

Definizione 2.8.1 (Cumulative Gain a Rango p). Dato il vettore delle rilevanze graduate $\mathbf{r} = [r_1, r_2, \dots, r_p]$, il Cumulative Gain è la somma lineare dei guadagni:

$$CG_p = \sum_{i=1}^p r_i \quad (2.17)$$

Il CG_p è insensibile alla posizione: scambiare un documento altamente rilevante al rango 1 con uno al rango 10 non altera il valore di CG_p .

Definizione 2.8.2 (Discounted Cumulative Gain (Formulazione Standard)). Il DCG applica uno sconto logaritmico basato sul rango della posizione i :

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)} \quad (2.18)$$

Significato Comportamentale dello Sconto Logaritmico

Lo sconto $\frac{1}{\log_2(i)}$ riflette il decadimento esponenziale dell'attenzione visiva dell'utente secondo il modello di lettura a cascata:

- Al rango $i = 4$, il fattore di sconto è $\frac{1}{\log_2(4)} = \frac{1}{2}$: il valore informativo percepito si dimezza rispetto alla prima posizione.
- Al rango $i = 8$, il fattore è $\frac{1}{\log_2(8)} = \frac{1}{3}$.
- Al rango $i = 16$, il fattore scende a $\frac{1}{\log_2(16)} = \frac{1}{4}$.

Nei motori di ricerca commerciali (Google, Bing) si impiega frequentemente la **formulazione esponenziale**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (2.19)$$

Chiarimento sulla Notazione degli Indici

Si noti la differenza nell'argomento del logaritmo:

- Nell'Equazione (2.18) il primo termine r_1 viene estratto per evitare $\log_2(1) = 0$ al denominatore, mentre la sommatoria parte da $i = 2$ con $\log_2(i)$.
- Nell'Equazione (2.19) la sommatoria include direttamente il rango $i = 1$ adottando $\log_2(i + 1)$ al denominatore ($\log_2(1 + 1) = \log_2(2) = 1$).
- Inoltre, il numeratore $2^{r_i} - 1$ enfatizza in modo non lineare i documenti di massima rilevanza ($r = 3 \implies 2^3 - 1 = 7$, mentre $r = 1 \implies 2^1 - 1 = 1$), penalizzando aspramente il loro mancato posizionamento nei primi ranghi.

2.8.2 Normalized Discounted Cumulative Gain (NDCG)

Poiché query differenti presentano un numero variabile di documenti rilevanti, il valore grezzo di DCG_p non è confrontabile tra query diverse. Si ricorre quindi all'**Ideal DCG** ($IDCG_p$), ottenuto calcolando il DCG della graduatoria ideale che ordina tutti i documenti per rilevanza decrescente:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1] \quad (2.20)$$

2.8.3 Esempio Numerico Completo: Calcolo Passo-Passo

Si consideri la seguente graduatoria di 10 documenti, valutati su scala 0–3:

$$\mathbf{r} = [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$$

La sequenza ideale ottenuta ordinando per rilevanza non crescente è:

$$\mathbf{r}^* = [3, 3, 3, 2, 2, 2, 1, 0, 0, 0]$$

La Tabella 2.4 riporta l'evoluzione dettagliata di ogni singolo termine posizionale.

i	r_i	Sconto	Guadagno Scontato	$DCG@i$	r_i^*	Guadagno Ideale	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	3.00	3	3.00	3.00	1.00
2	2	$\log_2(2) = 1.000$	2.00	5.00	3	3.00	6.00	0.83
3	3	$\log_2(3) = 1.585$	1.89	6.89	3	1.89	7.89	0.87
4	0	$\log_2(4) = 2.000$	0.00	6.89	2	1.00	8.89	0.76
5	0	$\log_2(5) = 2.322$	0.00	6.89	2	0.86	9.75	0.71
6	1	$\log_2(6) = 2.585$	0.39	7.28	2	0.77	10.52	0.69
7	2	$\log_2(7) = 2.807$	0.71	7.99	1	0.36	10.88	0.73
8	2	$\log_2(8) = 3.000$	0.67	8.66	0	0.00	10.88	0.80
9	3	$\log_2(9) = 3.170$	0.95	9.61	0	0.00	10.88	0.88
10	0	$\log_2(10) = 3.322$	0.00	9.61	0	0.00	10.88	0.88

Tabella 2.4: Tracciamento dettagliato del calcolo di $DCG@i$, $IDCG@i$ e $NDCG@i$ per $i = 1, \dots, 10$.

2.8.4 Confronto tra Funzioni di Ranking Alternative

Si consideri il caso studio di 4 documenti $\{d_1, d_2, d_3, d_4\}$ con gradi di rilevanza:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

- **Ground Truth (GT):** Ordinamento $\langle d_4, d_3, d_2, d_1 \rangle$ con vettori di rilevanza $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$

$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF_1):** Ordinamento $\langle d_3, d_4, d_2, d_1 \rangle$ con rilevanze $[2, 2, 1, 0]$. Poiché i primi due documenti possiedono il medesimo grado ($r = 2$), l'inversione non produce alcuna penalità posizionale:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$

$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordinamento $\langle d_3, d_2, d_4, d_1 \rangle$ con rilevanze $[2, 1, 2, 0]$. In questo caso, il documento d_2 ($r = 1$) scavalca erroneamente d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + \frac{2}{1.585} = 4.2619$$

$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

Questo calcolo dimostra con precisione matematica come la metrica NDCG penalizzi le inversioni di rango proporzionalmente alla gravità dello scambio di rilevanza.

Capitolo 3

Efficienza Algoritmica sulle Architetture CPU Moderne

3.1 Il Ruolo dell'Efficienza nell'Information Retrieval

Lo sviluppo di sistemi di elaborazione delle informazioni si articola lungo due dimensioni complementari e spesso in tensione tra loro:

1. **Efficacia (*Effectiveness*):** massimizza la soddisfazione dell'utente finale (*user satisfaction*), concentrandosi sulla qualità semantica dei risultati, sul ranking e sull'elaborazione del linguaggio naturale, tipicamente con strumenti ad alto livello di astrazione (Python, framework di Machine Learning).
2. **Efficienza (*Efficiency*):** minimizza i costi operativi infrastrutturali, progettando algoritmi ottimizzati fino all'ultimo ciclo di clock della CPU e all'ultimo bit di memoria occupato.

Non è richiesto che una singola figura professionale padroneggi entrambi gli estremi; tuttavia chi sviluppa ad alto livello deve comprendere i vincoli fisici dell'hardware sottostante per non progettare soluzioni impraticabili, mentre chi ottimizza a basso livello deve preservare la struttura e la semantica dei dati per non comprometterne l'efficacia.

3.1.1 I Tre Pilastri dell'Efficienza nei Sistemi IR

Esperienza utente. Nei sistemi web su scala planetaria, la risposta a una query deve essere erogata entro una frazione infinitesima di secondo. L'utente

percepisce qualsiasi ritardo superiore a pochi decimi di secondo: se i risultati tardano o risultano irrilevanti, abbandona il servizio a favore della concorrenza. L'efficienza algoritmica non serve solo a rispondere più rapidamente, ma consente di eseguire algoritmi di ranking e filtraggio più sofisticati ed efficaci *entro il medesimo budget temporale prestabilito* (tipicamente un tetto massimo di 100–200 ms).

Scalabilità operativa. I sistemi moderni gestiscono miliardi di documenti, elementi multimediali, query e log generati ogni secondo. Il numero di macchine necessarie è approssimativamente:

$$\text{Numero di macchine} = \left\lceil \frac{\text{Volume totale dei dati}}{\text{Capacità operativa del singolo nodo}} \right\rceil.$$

Se il volume dei dati raddoppia, un sistema ben progettato richiede al più il doppio delle macchine (scalabilità lineare); un algoritmo inefficiente può invece richiedere una crescita superlineare dei costi (ad es. quadruplicare i server), insostenibile su larga scala. Per rispettare le latenze richieste, i dati attivi devono risiedere in RAM ed evitare gli accessi a disco: ciò richiede algoritmi efficienti non solo in tempo ma anche in spazio, tramite tecniche di compressione. Inoltre, poiché i dati vengono replicati su più macchine per tolleranza ai guasti (*fault tolerance*), ogni inefficienza di memoria viene amplificata dal fattore di replica dell'infrastruttura.

Sostenibilità ambientale ed economica. La riduzione del numero di macchine fisiche abbatte direttamente il consumo energetico dei data center. I server ad alta densità dissipano ingenti quantità di calore e gran parte dell'energia è consumata dagli impianti di raffreddamento; per questo molti data center su larga scala vengono collocati in regioni fredde o vicino a corsi d'acqua (*free cooling*). Un risparmio computazionale marginale, anche solo del 3%, genera risparmi economici rilevanti e riduce l'impronta di carbonio.

3.2 La Gerarchia di Memoria e i Principi di Località

3.2.1 Il Divario di Latenza tra CPU e Memoria

Nel modello teorico computazionale classico (macchina RAM astratta) l'accesso a qualunque locazione di memoria ha costo unitario costante, $\mathcal{O}(1)$. Nelle macchine fisiche reali le unità aritmetiche della CPU operano a velocità

enormemente superiori rispetto alla memoria principale (DRAM), ed esiste una gerarchia di livelli intermedi (le *cache*) che approssima l'accesso rapido per i dati usati di frequente.

Livello hardware	Latenza tipica	Cicli CPU	Capacità tipica
Istruzione CPU (ALU)	0.5–5 ns	1–2	registri interni
L1I / L1D Cache	0.5 ns	≈ 1	32 KB per core
L2 Cache	1.5 ns	≈ 3 –4	256 KB (fino a 2 MB)
L3 Cache (LLC)	5.0 ns	≈ 10 –15	16 MB (fino a 64 MB+)
RAM principale (DRAM)	80.0 ns	≈ 160 –200+	64 GB+
Disco / Storage	500 000 ns (0.5 ms)	milioni	8 TB+

Tabella 3.1: Confronto quantitativo di latenza e capacità tra i livelli della gerarchia di memoria.

Definizione 3.2.1 (Algoritmo *Memory-Bound*). Un algoritmo che accede alla RAM centrale in modo disordinato paga ≈ 80 ns per ogni accesso, intervallo in cui la CPU potrebbe eseguire centinaia di istruzioni. L'algoritmo si dice **memory-bound**: il tempo di esecuzione è dominato dall'attesa del trasferimento dati dalla DRAM e la CPU trascorre la maggior parte del tempo in stallo, rendendo irrilevante l'ottimizzazione del numero di operazioni aritmetiche.

3.2.2 Architettura Fisica sul Chip

La cache L1 è integrata a ridosso dell'unità di decodifica e delle unità di esecuzione; per evitare conflitti di bus è fisicamente suddivisa in **L1I** (*Instruction Cache*, contiene il codice macchina prelevato per l'esecuzione) e **L1D** (*Data Cache*, contiene gli operandi delle istruzioni di `load/store`). La cache L2 è integrata nel singolo core e funge da cuscinetto a bassa latenza; la cache L3 è esterna ai singoli core ma interna al package ed è condivisa tra tutti i core della macchina.

La Cache Line come Unità Minima di Trasferimento

La RAM e la gerarchia di cache non trasferiscono mai singoli byte o singole variabili isolate: l'unità minima di trasferimento hardware è la **cache line**, tipicamente $B = 64$ byte. Se un programma necessita di un intero a 64 bit (8 byte) situato all'indirizzo A , l'hardware carica forzatamente l'intero blocco allineato

$$[\lfloor A/64 \rfloor \times 64, \lfloor A/64 \rfloor \times 64 + 63].$$

Se il programma legge quell'unico valore e poi salta a un indirizzo casuale, i restanti 56 byte (87.5% del traffico di memoria) vengono sprecati.

3.2.3 Località Temporale e Spaziale

Un programma sfrutta efficacemente la gerarchia di memoria solo se rispetta almeno una delle due forme di località:

- **Località temporale:** quando una data locazione di memoria viene acceduta, è altamente probabile che la medesima locazione venga acceduta nuovamente nell'immediato futuro. Le politiche di rimpiazzo delle cache (varianti di **LRU**, *Least Recently Used*) scartano i dati meno recentemente utilizzati per preservare le locazioni accedute di frequente.
- **Località spaziale:** quando una data locazione di memoria viene acceduta, è altamente probabile che locazioni contigue o vicine vengano accedute nell'immediato futuro. Poiché l'hardware trasferisce l'intera cache line, l'accesso a elementi contigui ammortizza il costo di caricamento su tutti gli elementi contenuti al suo interno.

3.2.4 Hardware Prefetcher

I processori integrano circuiti dedicati, gli **hardware prefetcher**, il cui compito è anticipare le richieste di memoria dell'applicazione:

- **Stream prefetcher:** se un programma scansiona linearmente la memoria da sinistra verso destra ($A, A + 64, A + 128, \dots$), il circuito rileva il pattern sequenziale e ordina il caricamento preventivo delle cache line successive in L1/L2 prima ancora che il programma emetta l'istruzione di lettura, mascherando così la latenza.

- **Stride prefetcher:** intercetta accessi a passo costante (*stride*), ad esempio scansioni che campionano la memoria a intervalli fissi; riconosciuto il gradiente costante, carica selettivamente le cache line corrispondenti al passo di salto.
- **Pattern casuali:** di fronte a salti casuali e non deterministici (*pointer chasing*), nessun hardware prefetcher è in grado di anticipare gli accessi, causando lo stallo completo della pipeline a ogni lettura.

3.3 Caso di Studio: Scansione Sequenziale contro *Pointer Chasing*

Per dimostrare sperimentalmente l'impatto della località e del prefetching si confrontano due funzioni Rust operanti su un vettore v di dimensione N . Il vettore di ingresso rappresenta una **permutazione casuale con un singolo ciclo** $\pi \in S_N$.

Definizione 3.3.1 (Permutazione con un singolo ciclo). Una permutazione π dell'insieme $\{0, 1, \dots, N - 1\}$ contiene un singolo ciclo se e solo se, partendo dall'indice 0, la catena $k_{t+1} = \pi(k_t)$ visita ogni posizione dell'array esattamente una volta prima di tornare a 0:

$$0 \rightarrow \pi(0) \rightarrow \pi(\pi(0)) \rightarrow \dots \rightarrow \pi^{N-1}(0) \rightarrow 0,$$

senza cicli isolati o sottocicli disgiunti.

Entrambe le funzioni calcolano la sommatoria di tutti gli elementi del vettore, $\sum_{i=0}^{N-1} v[i] = \frac{(N-1)N}{2}$.

Listing 3.1: Scansione sequenziale lineare.

```

1 pub fn sum(v: &[usize]) -> usize {
2     let mut sum = 0;
3     for &value in v.iter() {
4         sum += value;
5     }
6     sum
7 }
```

Listing 3.2: Scansione per salti casuali (*pointer chasing*).

```

1 pub fn sum_by_jumping(v: &[usize]) -> usize {
2     let mut sum = 0;
```

```

3   let mut k = 0;
4   for _ in 0..v.len() {
5       sum += v[k];
6       k = v[k]; // il valore letto diventa l'indice
                  della prossima iterazione
7   }
8   sum
9 }

```

Nel modello teorico RAM entrambe le funzioni eseguono N iterazioni con un'operazione di somma per passo: hanno cioè identica complessità asintotica $\mathcal{O}(N)$. Sulla macchina fisica, misurando il tempo medio per singolo elemento al variare della dimensione dell'array, emergono però divergenze marcate.

N (elementi)	Dimensione	Livello	sum()	sum_by_jumping()	Rallentamento
256	2 KiB	L1 (32 KB)	0.1 ns	1.3 ns	13×
8 192	64 KiB	L2 (256 KB)	0.1 ns	2.5 ns	25×
262 144	2 MiB	L3 (16 MB)	0.2 ns	13.4 ns	67×
16 777 216	128 MiB	RAM (DRAM)	0.4 ns	84.9 ns	212×

Tabella 3.2: Tempo medio per elemento di `sum()` e `sum_by_jumping()` al variare della dimensione dell'array.

3.3.1 Analisi delle Cause Fisiche

Perché `sum()` varia solo da 0.1 a 0.4 ns? Con interi `usize` a 64 bit (8 byte), in una cache line da 64 byte alloggiano $64/8 = 8$ elementi contigui: si ha piena località spaziale. L'accesso a una nuova linea comporta un miss iniziale, ma i successivi 7 accessi avvengono a costo pressoché nullo da cache, e lo stream prefetcher carica le linee successive in anticipo rispetto all'esecuzione. Indicando con T_{comp} il tempo di calcolo e con T_{load} il tempo di trasferimento ammortizzato, il costo osservato per elemento segue

$$\text{Latenza osservata} \approx \max(T_{\text{comp}}, T_{\text{load_ammortizzato}}),$$

quasi interamente mascherato dal calcolo; l'aumento da 0.1 a 0.4 ns su 128 MiB è dovuto unicamente alla saturazione della banda del bus verso la RAM.

Perché `sum_by_jumping()` arriva a 84.9 ns? Poiché la permutazione è casuale, l'indirizzo $k_{t+1} = v[k_t]$ punta a una locazione arbitraria: la probabilità che cada nella stessa cache line è trascurabile, quindi per ogni elemento letto

da 8 byte i restanti 56 byte trasferiti vengono scartati (assenza totale di località spaziale). Non essendovi passo lineare o costante, il prefetching è inibito. Inoltre la CPU deve conoscere l'indice della lettura successiva prima di poterla emettere: si tratta di una **dipendenza seriale** che impedisce ogni sovrapposizione. Quando la dimensione dell'array supera la capienza della L3 (128 MiB $\gg$ 16 MB), ogni accesso diventa un cache miss globale verso la DRAM:

Tempo per elemento $\approx$ latenza di accesso fisico alla DRAM $\approx$ 80–85 ns, valore che coincide esattamente con i 84.9 ns misurati.

Esempio 3.3.1 (Ammortizzare il costo del *pointer chasing* sfruttando la cache line). Se il salto avviene tra blocchi di 64 byte (8 interi a 64 bit), ma tutti gli elementi del blocco vengono sommati prima del salto successivo, il costo di un cache miss verso la RAM ($\approx$ 85 ns) viene sostenuto una sola volta ogni 8 operazioni di somma:

```

1  pub fn sum_by_jumping_cache_lines(v: &[usize]) -> usize
2      {
3          let mut sum = 0;
4          let mut k = 0;
5          let num_blocks = v.len() / 8;
6          for _ in 0..num_blocks {
7              let base = k * 8;
8              // sfrutta appieno la cache line gia' caricata
9              // in memoria
10             sum += v[base] + v[base + 1] + v[base + 2] + v[
11                 base + 3]
12                 + v[base + 4] + v[base + 5] + v[base + 6];
13             k = v[base + 7] % num_blocks; // puntatore al
14             // prossimo salto
15         }
16         sum
17     }

```

La latenza viene ammortizzata su 8 elementi, portando il tempo medio atteso per valore a

$$\text{Tempo medio per valore} \approx \frac{85 \text{ ns}}{8} \approx 10.6 \text{ ns},$$

un risultato intermedio tra la scansione lineare e il salto casuale elemento per elemento.

Nota 3.3.1 (Eliminare il rallentamento residuo di `sum()`). Il lieve rallentamento da 0.1 a 0.4 ns osservato in `sum()` si verifica perché $T_{\text{comp}} \approx 0.1 \text{ ns}$ è

inferiore al tempo necessario ad alimentare il flusso dati dalla RAM su vettori molto grandi (regime memory-bound). Aumentando la complessità computazionale del corpo del ciclo (ad es. moltiplicazioni o funzioni non banali su ciascun elemento) si ottiene $T_{\text{comp}} \gg T_{\text{load_RAM}}$: la relazione $\max(T_{\text{comp}}, T_{\text{load}})$ diventa costantemente pari a T_{comp} , il regime diventa *CPU-bound* e la curva dei tempi risulta perfettamente piatta al variare di N .

3.4 Forme di Parallelismo e Architettura della Pipeline

Le moderne CPU sfruttano molteplici forme di parallelismo: il **multi-threading/multi-core** (più core fisici indipendenti sullo stesso processore), il **pipelining** (suddivisione dell'esecuzione dell'istruzione in stadi operanti simultaneamente su istruzioni contigue) e l'esecuzione **superscalare** (unità di calcolo multiple nello stesso core per elaborare più istruzioni indipendenti per ciclo).

Parallelismo Triviale nei Motori di Ricerca

Nei motori di ricerca non è necessario sincronizzare più core per rispondere a una singola query: si adotta il **parallelismo triviale** (*embarrassingly parallel*), in cui, data la presenza di migliaia di utenti simultanei, ogni thread su un singolo core serve in modo completamente isolato una specifica query. Non essendovi condivisione di stato, si azzerano le contese sui lock e i problemi di coerenza della cache.

3.4.1 Anatomia del Pipelining a Cinque Stadi

Il pipelining scompone l'esecuzione di un'istruzione in stadi sequenziali indipendenti che, in un'architettura standard, sono cinque:

1. **IF** (*Instruction Fetch*): prelievo dell'istruzione in codice macchina dalla memoria (L1I) tramite il *program counter*.
2. **ID** (*Instruction Decode*): decodifica dell'istruzione in micro-operazioni (μ -ops) e lettura dei registri sorgente.
3. **EX** (*Execute*): esecuzione del calcolo aritmetico-logico nelle ALU o calcolo dell'indirizzo effettivo di memoria.
4. **MEM**: accesso al sottosistema di memoria (L1D) per istruzioni di caricamento o scrittura (LOAD/STORE).

5. **WB** (*Write-Back*): scrittura del risultato nei registri architetturali e ritiro dell'istruzione.

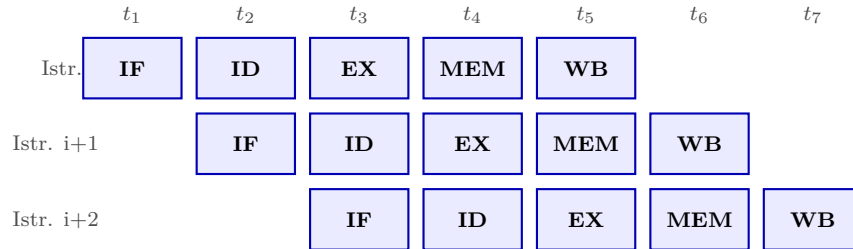


Figura 3.1: Sovrapposizione degli stadi della pipeline a 5 stadi su istruzioni consecutive.

3.4.2 Latenza, Throughput e Profondità della Pipeline

Sia k il numero di stadi della pipeline e τ il periodo del ciclo di clock. La **latenza** L è il tempo totale richiesto da una singola istruzione per attraversare l'intera pipeline, $L = k \cdot \tau$: nelle CPU moderne la latenza per un'operazione semplice varia tipicamente tra 10 e 20 cicli di clock. Il **throughput** W è il numero di istruzioni completate per unità di tempo e, a regime, $W = 1/\tau$ istruzioni al secondo, poiché a ogni ciclo un'istruzione esce dalla pipeline. Il periodo τ è imposto dal ritardo dello stadio più lento della catena circuitale,

$$\tau \geq \max_{1 \leq i \leq k} (t_{\text{stadio}_i}).$$

Per innalzare la frequenza di clock (ridurre τ) i progettisti aumentano la profondità k della pipeline (fino a 15–20 o più stadi nelle CPU moderne), implementando ciascuno stadio con circuiteria più semplice e veloce; di contro, l'aumento di k dilata la penalità in caso di svuotamento forzato della pipeline.

3.5 Esecuzione Superscalare, Interleaving e Conflitti Strutturali

Un processore si definisce **superscalare** quando possiede l'abilità architetturale di avviare e completare più di un'istruzione per ciclo di clock ($IPC > 1$, *Instructions Per Cycle*) all'interno dello stesso core, integrando unità logico-aritmetiche multiple (ALU per interi, moltiplicatori, unità di shift, unità di

load/store) servite da porte di emissione parallele (*execution ports*). Architetture reali come Intel Haswell dispongono ad esempio di una coda di decodifica a 56 voci, un *reorder buffer* a 192 voci, una stazione di prenotazione unificata a 60 voci e 8 porte di esecuzione parallele, ciascuna specializzata per classi di operazioni (ALU intere, load/store, unità vettoriali, branch, ecc.).

3.5.1 L'Esperimento dell'*Interleaving* Aritmetico

Si sottopongono a benchmark quattro funzioni Rust che operano su un vettore di interi a 64 bit `&[u64]`, accumulando un numero crescente di operazioni indipendenti per iterazione.

Listing 3.3: Da un'unica riduzione a quattro riduzioni indipendenti per iterazione.

```
1 pub fn do_stuff(v: &[u64]) -> u64 {
2     let mut prod = 1;
3     for &value in v.iter() {
4         prod *= value;
5     }
6     prod
7 }
8
9 pub fn do_more_stuff(v: &[u64]) -> (u64, u64) {
10    let mut prod = 1;
11    let mut sum = 0;
12    for &value in v.iter() {
13        prod *= value;
14        sum += value;
15    }
16    (prod, sum)
17 }
18
19 pub fn do_more_stuff_2(v: &[u64]) -> (u64, u64, u64) {
20    let mut prod = 1;
21    let mut sum = 0;
22    let mut xor = 0;
23    for &value in v.iter() {
24        prod *= value;
25        sum += value;
26        xor ^= value;
27    }
28    (prod, sum, xor)
29 }
```

```

30
31 #[inline]
32 pub fn do_more_stuff_3(v: &[u64]) -> (u64, u64, u64, u64) {
33     let mut prod = 1;
34     let mut sum = 0;
35     let mut xor = 0;
36     let mut or = 0;
37     for &value in v.iter() {
38         prod *= value;
39         sum += value;
40         xor ^= value;
41         or |= value;
42     }
43     (prod, sum, xor, or)
44 }

```

Funzione	Tempo per iterazione
do_stuff() (1 operazione)	0.4 ns
do_more_stuff() (2 operazioni)	0.4 ns
do_more_stuff_2() (3 operazioni)	0.4 ns
do_more_stuff_3() (4 operazioni)	0.5 ns

Tabella 3.3: Tempo per iterazione al crescere del numero di riduzioni indipendenti accumulate.

3.5.2 Sovrapposizione delle Istruzioni e Saturazione delle Porte

Da una a tre operazioni il tempo resta invariato a 0.4 ns perché le variabili di accumulo non presentano dipendenze reciproche: $\text{prod}_{t+1} = \text{prod}_t \cdot v[i]$, $\text{sum}_{t+1} = \text{sum}_t + v[i]$, $\text{xor}_{t+1} = \text{xor}_t \oplus v[i]$. Il core superscalare invia contemporaneamente la moltiplicazione, l'addizione e lo XOR a tre porte di esecuzione distinte: le tre operazioni vengono completate nel medesimo ciclo di clock, e il lavoro aggiuntivo è svolto a costo nullo.

Con la quarta operazione (**or**) il tempo sale a 0.5 ns anziché raddoppiare a 0.8 ns: si tratta di uno **structural hazard**, poiché le unità fisiche disponibili per calcoli logico-aritmetici in parallelo in un singolo ciclo sono sature (solo tre istruzioni di quel tipo possono essere emesse simultaneamente), e una delle quattro operazioni deve attendere. Tuttavia la CPU *out-of-order* non processa

le iterazioni come compartimenti stagni: srotola implicitamente la finestra di istruzioni raggruppandole a blocchi di tre, assorbendo l'operazione in eccesso nei cicli successivi e saturando le capacità residue delle porte. L'overhead reale corrisponde quindi a una frazione marginale di ciclo ($\Delta t = +0.1 \text{ ns}$) anziché a un ciclo intero aggiuntivo.

3.6 Data Hazard e Dipendenze Seriali

Definizione 3.6.1 (Data Hazard). Si verifica un **data hazard** quando un'istruzione necessita di un operando non ancora disponibile poiché in fase di calcolo da parte di un'istruzione precedente. Corrisponde a una dipendenza sui dati di tipo **RAW** (*Read-After-Write*):

$$\text{Inst}_A : R_1 \leftarrow R_2 + R_3 \quad \Longrightarrow \quad \text{Inst}_B : R_4 \leftarrow R_1 \times R_5,$$

dove Inst_B deve rimanere in stallo nella pipeline finché Inst_A non conclude il calcolo.

Si consideri un codice che esegue lo stesso numero di operazioni per iterazione della precedente `do_more_stuff_3`, ma concatenando i dati in una catena stretta:

Listing 3.4: Catena di dipendenza seriale RAW tra quattro operazioni.

```

1 pub fn do_more_stuff_4(v: &[u64]) -> (u64, u64, u64) {
2     let mut prod = 1;
3     let mut sum = 0;
4     let mut xor = 0;
5     for &value in v.iter() {
6         sum += value; // 1. calcola sum
7         prod *= sum; // 2. dipende strettamente da sum
8         xor ^= prod; // 3. dipende strettamente da
9                       prod
9         sum += xor; // 4. dipende da xor e
10                    sovrascrive sum per l'iterazione successiva!
11     }
12     (prod, sum, xor)
13 }
```

A causa della catena RAW, le unità di esecuzione non possono avviare alcun calcolo in parallelo:

$$\text{Latenza iterazione} = t_{\text{add}} + t_{\text{mul}} + t_{\text{xor}} + t_{\text{add_feedback}}.$$

Poiché l'ultima operazione aggiorna `sum` e la prima istruzione dell'iterazione successiva richiede il nuovo valore di `sum`, la CPU *out-of-order* non può avviare l'iterazione $t + 1$ in anticipo: le unità superscalari restano inutilizzate. Sperimentalmente, contro i 0.5 ns di `do_more_stuff_3` (operazioni indipendenti), `do_more_stuff_4` misura 1.7 ns per iterazione ($\approx 3.5\times$ più lento), pur eseguendo lo stesso numero di operazioni aritmetiche.

3.6.1 Soluzione Algoritmica: *Interleaving* di Vettori Indipendenti

Quando occorre eseguire la medesima funzione con dipendenza stretta su due vettori distinti `V` e `West`, l'approccio sequenziale tradizionale (`do_more_stuff_4(V)` seguito da `do_more_stuff_4(West)`) impiega un tempo complessivo $2T$. Se invece si scrive un'unica funzione che elabora entrambi i vettori all'interno della stessa iterazione, alternando le due catene indipendenti:

Listing 3.5: Fusione (interleaving) di due catene RAW indipendenti tra loro.

```

1  for i in 0..n {
2      // catena per V (occupa le porte al ciclo k)
3      sum_v += v[i];
4      prod_v *= sum_v;
5      xor_v ^= prod_v;
6      sum_v += xor_v;
7      // catena per West, completamente indipendente da V!
8      sum_w += west[i];
9      prod_w *= sum_w;
10     xor_w ^= prod_w;
11     sum_w += xor_w;
12 }
```

il tempo di esecuzione misurato è identico a quello del solo vettore `V` ($\approx T$, anziché $2T$). Le operazioni su `West` non dipendono da quelle su `V`: mentre la catena di `V` è in attesa dei risultati delle ALU, le unità di calcolo libere processano la catena di `West`, saturando a costo nullo i tempi morti imposti dalle dipendenze sui dati.

3.7 Control Hazard, Esecuzione Speculativa e Branch Prediction

Definizione 3.7.1 (Control Hazard). Si verifica un **control hazard** quando la CPU non può determinare quale sia la successiva istruzione da prelevare dalla

memoria (*instruction fetch*) a causa della presenza di un salto condizionato (*if, branch*).

La CPU non può fermarsi a ogni condizione: adotta pertanto l'**esecuzione speculativa**, ipotizzando tramite un *branch predictor* quale ramo verrà imboccato e prelevando speculativamente le istruzioni successive per alimentare la pipeline. Se l'esito della condizione si rivela discorde rispetto alla predizione, tutto il lavoro speculativo completato negli stadi intermedi deve essere annullato, con uno svuotamento completo della pipeline (*pipeline flush*) e il ricaricamento dal ramo corretto:

Penalità per mispredizione = 15–20 cicli di clock.

3.7.1 Il Paradosso della Condizione Casuale

Si introduce un filtro condizionale in un codice con catena di dipendenza, operando su un array di interi casuali uniformemente distribuiti in $[0, 99]$:

Listing 3.6: Ramo condizionale su valori casuali: la CPU non può predire l'esito.

```
1 pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {
2     let mut prod = 1;
3     let mut sum = 0;
4     let mut xor = 0;
5     for &value in v.iter() {
6         if value > 50 { // vera con p = 0.5 su valori
7             sum += value;
8             prod *= sum;
9             xor ^= prod;
10            sum += xor;
11        }
12    }
13    (prod, sum, xor)
14 }
```

Poiché la condizione è vera in media il 50% delle volte, ci si aspetterebbe intuitivamente un tempo pressoché dimezzato rispetto a `do_more_stuff_4` ($1.7/2 \approx 0.85$ ns). Il tempo misurato sale invece a 3.5 ns, più del doppio più lento della versione priva di condizione.

Il Costo Reale non è il Calcolo ma la Mispredizione

Con valori casuali il branch predictor ha accuratezza pari al lancio di una moneta, $P(\text{hit}) = P(\text{miss}) = 0.50$. Il costo medio per iterazione è allora

Costo medio $\approx 0.5 \times T_{\text{corpo}} + P(\text{miss}) \times \text{Penalità flush} \approx 0.5 \times 0.85 \text{ ns} + 0.5 \times 5.0 \text{ ns} \approx 2.9\text{--}3.5 \text{ ns}$,

avendo assunto una penalità di 15–20 cicli ($\approx 4\text{--}5 \text{ ns}$ su una CPU a 4 GHz). Il costo dell'algoritmo non è più dettato dalle operazioni aritmetiche, ma è dominato dal tempo perso a svuotare e ricaricare la pipeline.

3.7.2 Verifica Sperimentale: Ordinamento dei Dati e Pattern Periodici

Per dimostrare che il rallentamento deriva esclusivamente dal comportamento del branch predictor, si esegue lo stesso identico codice modificando solo la disposizione dei dati nell'array.

Configurazione	Tempo per iterazione	Comportamento del predittore
<code>do_more_stuff_4()</code> (nessun <code>if</code>)	1.7 ns	—
<code>do_more_stuff_5()</code> , dati casuali	3.5 ns	ramo imprevedibile, miss rate $\approx 50\%$
<code>do_more_stuff_5()</code> , dati ordinati	1.2 ns	predizione quasi perfetta, un solo mis
<code>do_more_stuff_5()</code> , dati alternati	1.1 ns	sequenza periodica, miss rate $\approx 0\%$

Tabella 3.4: Tempo per iterazione di `do_more_stuff_5()` al variare della disposizione dei dati.

Con **dati ordinati** (tutti gli elementi ≤ 50 all'inizio dell'array e tutti quelli > 50 alla fine) il predittore assume che il salto non sia preso per l'intera prima metà e che sia preso per la seconda: si verifica un unico misprediction al punto di transizione, e rimosso il costo dei flush il tempo scende a 1.2 ns, inferiore ai 1.7 ns della versione senza condizione poiché metà delle iterazioni non esegue il corpo dell'`if`. Con **dati alternati** secondo un pattern periodico (*falso, vero, falso, vero, ...*), i predittori integrano tabelle storiche dei pattern (*branch history shift register*): identificata la periodicità, il predittore anticipa l'esito a ogni passo senza errori, annullando gli stalli.

3.7.3 Tecniche *Branchless* e il Salto Implicito dei Cicli `for`

Se i dati non sono ordinabili né predicibili, l'unica soluzione consiste nel rimuovere fisicamente il salto condizionale (*branchless code*), sostituendolo con espressioni algebriche o istruzioni condizionali assembly (`CMOV`). Ad esempio, invece di

```
1 if value > 50 {  
2     sum += value;  
3 }
```

si mappa la guardia logica su un valore binario scalare (0 o 1), eseguendo sempre l'operazione aritmetica:

```
1 let condition = (value > 50) as u64; // vale 1 se vero,  
    0 se falso  
2 sum += value * condition;           // somma 'value' se  
    vero, somma '0' se falso
```

Il programma compie sempre il calcolo matematico, ma il flusso di istruzioni è lineare: non vi sono salti, e le mispredizioni sono azzerate.

Nota 3.7.1 (Perché il salto del ciclo `for` non costituisce un problema). Ogni ciclo `for` si conclude a livello macchina con un salto condizionato all'inizio del blocco. Questo branch non degrada le prestazioni perché è altamente asimmetrico: per N iterazioni la condizione di permanenza nel ciclo risulta vera per $N - 1$ volte consecutive ed è falsa solo all'uscita. Il branch predictor impara immediatamente il pattern ("salto preso") e commette un unico errore di predizione al termine del ciclo, con un impatto del tutto trascurabile sul tempo complessivo.

3.8 Sintesi: dai Vincoli Hardware alle Scelte Algoritmiche

I fenomeni analizzati in questo capitolo – gerarchia di memoria e località, pipelining, esecuzione superscalare, *data hazard* e *control hazard* – non sono dettagli implementativi trascurabili, ma vincoli fisici che determinano il reale comportamento prestazionale degli algoritmi di Information Retrieval su larga scala. In particolare:

- le strutture dati (posting list, indici invertiti) vanno progettate privilegiando l'accesso sequenziale e contiguo, per sfruttare la località spaziale e l'hardware prefetching;

- quando possibile, conviene esprimere più riduzioni indipendenti all'interno dello stesso ciclo per saturare le porte di esecuzione superscalari a costo marginale;
- le catene di dipendenza strette (RAW) tra variabili di accumulo vanno identificate e, se possibile, disaccoppiate o intrecciate (*interleaving*) con calcoli indipendenti;
- i rami condizionali imprevedibili sui dati vanno eliminati con tecniche *branchless* o mitigati riordinando i dati per favorire la predizione.

La trattazione della vettorizzazione SIMD e degli strumenti di analisi del codice compilato (*Compiler Explorer*) è rimandata a un capitolo successivo.

Bibliografia

- [1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.
- [3] Alistair Moffat and Luke Stuvier. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.
- [4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.
- [5] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.
- [7] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.
- [8] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.